Digit Recognition Project - Team Summary

Goal: Learn computer vision by locating one printed digit (0-9) on a simulated A4 page, drawing its bounding box, and recognizing its value.

Dataset

Split	Images	Per digit
Training	3,500	350
Validation	500	50
Test	1,000	100
Total	**5,000**	**500**

• One grayscale PNG per page, 595 × 842 pixels, with an upright black digit on white paper.

• Random position, size, font, and stroke thickness; digits stay fully inside the page.

• Separate annotation CSVs contain digit labels and tight bounding boxes. Source images have no drawn boxes.

• Colab-ready Python generates the folders and ZIP. All 5,000 images were checked for class balance, duplicates, and bounding-box containment.

Planned approach

Threshold → find contours → bounding box → crop and normalize → HOG features → SVM prediction.

OpenCV (cv2) provides the image-processing and classification tools; NumPy handles arrays. HOG describes local edge directions. The SVM learns to associate those features with digits. **Only the SVM requires training; no neural network is needed.**

Next steps

Implement the pipeline, train on training data, tune on validation, and evaluate the final fixed pipeline on test. Measure digit accuracy, a confusion matrix, and bounding-box intersection over union (IoU). End-to-end testing must use predicted boxes.

Current status: Generator and dataset complete; detection and recognition remain to be implemented. Start with this clean synthetic baseline, then add color, rotation, lighting, noise, or real images. Synthetic test accuracy alone does not establish real-world performance.